

Disciplina: Teste de Software

Refatoração de Testes e Detecção de Test Smells

Aluno: Carlos José Gomes Batista Figueiredo

Matrícula: 855198

Repositório: <https://github.com/CarlosJFigueiredo/test-smelly>

1. Análise de Smells

A suíte original (test/userService.smelly.test.js) foi lida manualmente e depois analisada com ESLint. Abaixo, três dos smells encontrados.

1.1 Lógica condicional (for + if/else) e expects condicionais

O teste de desativação percorre uma lista com for e decide com if/else qual asserção executar. **Por que é um mau cheiro:** o teste passa a ter caminhos próprios, que precisam ser entendidos e podem conter bugs; vários cenários (comum e admin) ficam misturados em um só, e a falha não diz qual deles quebrou. **Risco:** se a condição mudar, uma asserção deixa de rodar sem que ninguém perceba, e o teste continua verde.

1.2 Try/catch sem falha garantida (falso positivo)

O teste do menor de idade põe o expect dentro do catch. **Por que é um mau cheiro:** se a exceção nunca for lançada, o catch não executa e nenhuma asserção roda. **Risco:** se a validação de idade for removida do código, o teste continua passando e esconde o bug. Fiz esse experimento (seção 4): a suíte smelly não detectou a mutação.

1.3 Teste frágil (acoplado ao formato exato)

O teste do relatório compara a linha completa ID: ..., Nome: Alice, Status: ativo\n e o prefixo do título. **Por que é um mau cheiro:** verifica a implementação (formatação) e não o comportamento (o relatório lista os usuários). **Risco:** qualquer ajuste de espaço, ordem ou pontuação quebra o teste sem haver bug, o que gera custo de manutenção e desconfiança na suíte.

Outros smells encontrados: **Eager Test** (criar e buscar no mesmo teste, com dois Acts), **teste desabilitado** (test.skip vazio) e **dados soltos** (strings repetidas e um objeto padrão usado em um só teste).

2. Processo de Refatoração

Escolhi o teste *'deve desativar usuários se eles não forem administradores'*, que concentra lógica condicional, expects condicionais, vários cenários em um teste e nome pouco preciso.

Antes (smelly)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin];
  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (clean)

```
describe('deactivateUser', () => {
  test('deve retornar true ao desativar usuário comum', () => {
    // Arrange
    const comum = criarUmUsuario({ isAdmin: false });
    // Act
    const resultado = userService.deactivateUser(comum.id);
    // Assert
    expect(resultado).toBe(true);
  });
  test('deve marcar usuário comum como inativo após desativação', () => { ... });
  test('deve retornar false ao tentar desativar administrador', () => {
    // Arrange
    const admin = criarUmUsuario({ isAdmin: true });
    // Act
    const resultado = userService.deactivateUser(admin.id);
    // Assert
  });
});
```

```

    expect(resultado).toBe(false);
  });
  test('deve manter administrador ativo após tentativa de desativação', () => { ... });
  test('deve retornar false ao desativar usuário inexistente', () => { ... });
});

```

Decisões: (1) removi o for e o if, e cada cenário virou um teste com uma única razão para falhar; (2) apliquei AAA com comentários explícitos, e o Act é sempre uma chamada; (3) criei a função `umUsuario(overrides)`, que deixa claro no teste só o dado relevante (`isAdmin`); (4) os nomes descrevem o comportamento esperado; (5) adicionei o cenário de id inexistente, antes não coberto. Nos demais testes: `expect(fn).toThrow(msg)` no lugar do try/catch, verificações com `stringContaining` no relatório (menos frágil), criação e busca separadas (Eager Test) e o `test.skip` virou um teste real do relatório vazio.

3. Relatório da Ferramenta (ESLint)

Configuração: `eslint@8 + eslint-plugin-jest`, com `.eslintrc.json` conforme o enunciado (fixei a v8 porque as versões mais novas ignoram esse formato de configuração). Saída da primeira execução de `npx eslint .:`

```

test\userService.smelly.test.js
 44:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning Tests should not be skipped      jest/no-disabled-tests
 77:3  warning Test has no assertions          jest/expect-expect

```

6 problems (4 errors, 2 warnings)

(Saída do terminal em formato texto; o arquivo `docs/eslint-primeira-execucao.txt` está no repositório.)

Comentário: o ESLint apontou automaticamente, em segundos, os smells estruturais: os três expects condicionais do if/else (linhas 44, 46, 49), o expect dentro do catch (linha 73) e o teste pulado e sem asserções (linha 77). Ele **não** encontra os smells semânticos (teste frágil, Eager Test, dados soltos), que exigem leitura humana; por isso as duas análises se complementam. Após a refatoração, `npx eslint test/userService.clean.test.js` retorna 0 erros e 0 avisos.

4. Validação

`npm test`: 2 suítes passando, 17 testes passando e 1 pulado (o `test.skip` do arquivo original, que foi preservado como pedido).

Teste de mutação manual: troquei `idade < 18` por `idade < 0` em `src/userService.js`. Resultado: a suíte **smelly continuou passando** (falso positivo do try/catch) e a suíte **clean falhou**, como deveria. O código foi restaurado em seguida.

5. Conclusão

Testes limpos funcionam como documentação executável e como rede de segurança confiável. Com AAA, um comportamento por teste e sem lógica condicional, uma falha aponta diretamente o problema. A mutação mostrou na prática que um teste verde pode não provar nada. A análise estática complementa o processo: automatiza a detecção dos smells estruturais, roda no CI e impede regressões na qualidade dos testes, enquanto a revisão humana cobre os smells semânticos. Juntas, as duas práticas reduzem o custo de manutenção e mantêm a suíte sustentável com a evolução do projeto.